

# Session 3

## Deep learning and representation

---

What you build today: Project 2, the semantic understanding engine.

# The job

---

- Tags, several per ticket, so "SSO" and "Outage" route differently at the same priority
- Similar past tickets in front of the agent before they type
- Priority again, so we can measure it against Project 1
- Exported to a runtime with no PyTorch in the container

# Memory arithmetic

---

| Item                      | Bytes per parameter |
|---------------------------|---------------------|
| Weights, fp32             | 4                   |
| Gradients                 | 4                   |
| Adam moments, two of them | 8                   |
| <b>Full fine-tuning</b>   | <b>16</b>           |

66M parameters times 16 bytes is about 1 GB before activations. "It does not fit" is usually the optimiser.

# LoRA changes the arithmetic

---

- Freeze the encoder
- Add small low-rank matrices to query and value projections
- Train those and the heads: under one percent of parameters
- Optimiser state becomes negligible; a laptop is enough

| Knob    | Start with                |
|---------|---------------------------|
| rank    | 8                         |
| alpha   | 16                        |
| targets | <code>q_lin, v_lin</code> |

# The input pipeline is the bottleneck

---

- Profile before blaming the accelerator
- Fixed-length padding spends most compute on padding tokens
- Dynamic padding to the longest in the batch is the biggest single saving
- A deadlocked dataloader worker looks exactly like a slow GPU

## Multi-label is not multi-class

---

- Tags do not compete: BCE per tag, not softmax across tags
- One threshold per tag, chosen on validation
- A flat 0.5 under-predicts every rare tag
- Priority stays single-label, weighted cross-entropy: Session 2's lesson in a new loss

## A training loop you can read

---

1. Forward under autocast when there is a GPU
2. Loss, scaled, backward
3. Accumulate gradients across small batches
4. Unscale, clip, step, update the scaler, step the schedule
5. Check the loss is finite. A NaN at step 40 is a learning rate problem

# Checkpoints that resume

---

What a resume needs:

- adapter and head weights
- optimiser moments
- scheduler position
- epoch, step, best score so far

Weights alone restart Adam cold. That is a different run.

## Export: the serving boundary

---

- ONNX: one graph, a small runtime, no PyTorch in the image
- Dynamic batch and sequence axes, or the graph accepts one shape
- Parity on held-out tickets within  $1e-3$  before you trust it
- Dynamic int8: linear weights as 8-bit integers, no calibration set, first thing to try on CPU

## Where ONNX leaks

---

- Custom operators
- Data-dependent control flow
- Anything the tracer turns into a constant
- Read the export warnings once. Then read them again after an upgrade

# The benchmark is the deliverable

---

| Model                | Macro-F1 | P0 recall | p50 ms | Size | USD per 1k |
|----------------------|----------|-----------|--------|------|------------|
| Project 1, linear    |          |           |        |      |            |
| Project 2, PyTorch   |          |           |        |      |            |
| Project 2, ONNX fp32 |          |           |        |      |            |
| Project 2, ONNX int8 |          |           |        |      |            |

You fill it in today. Sessions 5 and 6 route on it.

## Acceptance criteria

---

1. Trainable parameters under 2 percent, printed
2. Tag micro-F1 and priority macro-F1 clearly above chance on validation
3. Training resumes from a mid-run checkpoint with optimiser state
4. ONNX fp32 matches PyTorch within  $1e-3$  on held-out tickets; int8 is smaller
5. The benchmark table exists with measured latency for all four rows

`make session03` reports every test passed.

## Questions worth asking

---

- For which tickets is Project 2's extra latency worth paying?
- Where do the German tickets land in the error analysis?
- If the customer runs TensorFlow Serving, what here transfers unchanged?